

CFGS Desarrollo de aplicaciones web

Módulo profesional: Programación



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



Unió Europea

Fons Social Europeu

L'FSE inverteix en el teu futur



Material elaborado por:

Edu Torregrosa Llácer

(aulaenlanube.com)

Esta obra está licenciada bajo la licencia **Creative Commons**
Atribución-NoComercial-CompartirIgual 4.0 internacional. Para ver una
copia de esta licencia visita:
<https://creativecommons.org/licenses/by-nc-sa/4.0/>



Attribution-NonCommercial-ShareAlike
4.0 International (CC BY-NC-SA 4.0)

Herencia y excepciones en JAVA



1. Herencia en JAVA
 - a. Estructura de clases
 - b. Sobreescritura de métodos
 - c. Polimorfismo
 - d. Interfaces
 - e. Clases abstractas
2. Comparar objetos
3. Tratamiento de excepciones
 - a. Capturar excepciones
 - b. Lanzar excepciones
 - c. Propagar excepciones
 - d. Excepciones personalizadas

Introducción

La **Herencia** es uno de los 4 pilares de la programación orientada a objetos junto con la Abstracción, Encapsulación y Polimorfismo.

Hay 3 palabras reservadas "nuevas" como son "**extends**", "**protected**" y "**super**".

- **extends**: Esta palabra reservada, indica a la clase hija cuál va a ser su clase padre, es decir, de qué clase va a heredar tanto sus atributos como sus métodos. Una clase en JAVA puede extender a una única clase. Sin embargo, podemos tener todos los niveles de Herencia que necesitemos
- **protected**: sirve para indicar un tipo de visibilidad de los atributos y métodos de la clase padre y significa que cuando un atributo es 'protected' o protegido, solo es visible ese atributo o método desde una de las clases hijas y no desde otra clase.
- **super**: se utiliza para hacer referencia a la clase padre o a la superclase. Nos permite en la definición de una clase hija, acceder a los atributos y métodos de la clase padre. El uso más común es para llamar a un constructor de la clase padre desde el constructor de la clase hija, en ese caso la llamada deberá ser la primera instrucción del método.

Introducción

```
public class Padre {  
    String nombre;  
    public Padre(String nombre) {  
        this.nombre = nombre;  
        System.out.print(nombre);  
    }  
}  
  
public class Hija extends Padre {  
    int edad;  
    public Hija(int edad, String padre) {  
        super(padre);  
        this.edad = edad;  
        System.out.println(" tiene una hija de " + edad + " años");  
    }  
}
```

Introducción

```
public class Padre {
    String nombre;
    public Padre(String nombre) {
        this.nombre = nombre;
        System.out.print(nombre);
    }
    public void mostrarInfo() { System.out.println("Padre: " + nombre); }
}
public class Hija extends Padre {
    String nombre;
    int edad;
    public Hija(int edad, String padre, String hija) {
        super(padre);
        this.nombre = hija;
        this.edad = edad;
        System.out.println(" tiene una hija de " + edad + " años");
    }
    public void mostrarInfo() {
        System.out.println("Hija: " + nombre + ", edad: " + edad + " años");
        super.mostrarInfo();
    }
}
```

Ejemplo Herencia en JAVA :

Clase Persona

Herencia en JAVA

- Se pide construir la clase Estudiante con los atributos nombre, edad y créditos matriculados. Se dispone de la clase Persona ya implementada.

```
public class Persona{
    private String nombre;
    private int edad;
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    public String getNombre() { return this.nombre; }
    public String toString() { return "Nombre: " + this.nombre + " Edad:" + this.edad; }
}
```

- Opciones posibles:
 - **Inapropiada:** Ignorar la clase Persona y construir la clase Estudiante con tres atributos (edad, nombre y créditos). Se repite la declaración de atributos y métodos ya hecha en la clase Persona.
 - **Apropiada:** Utilizar la herencia para definir la clase Estudiante en base a la clase Persona.

Herencia en JAVA

Construcción de la clase derivada Estudiante a partir de la clase base Persona.

- La clase Estudiante hereda (puede utilizar) todos los atributos y métodos que no son privados en Persona.

```
public class Persona {  
    private String nombre;  
    private int edad;  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
    public String toString() { return "Nombre: " + this.nombre + " Edad:" + this.edad; }  
    public String getNombre() { return this.nombre; }  
}
```

```
public class Estudiante extends Persona {  
    private int creditos;  
    public Estudiante(String nombre, int edad) {  
        super(nombre, edad);  
        this.creditos = 60;  
    }  
    public int getCreditos() { return this.creditos; }  
}
```

Persona

extends

Estudiante

Herencia en JAVA

```
public class AppEstudiantePersona {  
    public static void main (String[] args) {  
        Estudiante e = new Estudiante("Luís García",20);  
        Persona p = new Persona("Marta Gómez", 20);  
        System.out.println(p.getNombre());  
        System.out.println(e.getNombre()+" : "+e.getCreditos()+" créditos");  
    }  
}
```

- Se pueden invocar los métodos declarados en Persona desde un objeto Estudiante ya que Estudiante hereda de Persona.
 - La clase Estudiante puede acceder a los atributos y los métodos no privados de la clase Persona.

p.getNombre() y **e.getNombre()** están accediendo al mismo método, pero lo hacen desde instancias de objetos distintos(Persona y Estudiante)

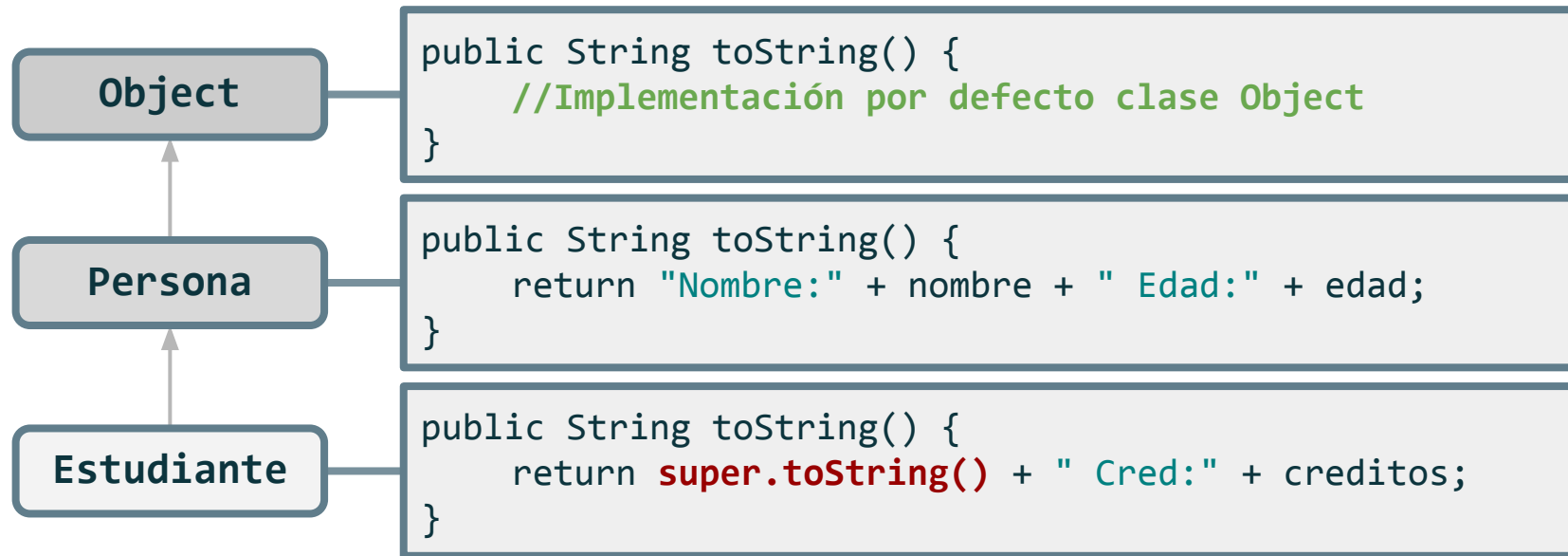
Sobreescritura de métodos en JAVA

Sobreescritura de métodos

- Todo método no privado de la clase base que se defina de nuevo en la clase derivada se sobreescribe:
- Sobreescritura **completa**:
 - Se define en la clase derivada un método:
 - Con el mismo perfil que en la base (nombre y lista de parámetros).
 - Con el mismo tipo de resultado que en la base.
- Sobreescritura **parcial**:
 - Cuando solamente se quiere cambiar parcialmente el comportamiento del método de la clase base. Se utiliza **super** para invocar el método de la clase base.

Sobreescritura de métodos

- Ejemplo:
 - Un ejemplo es la sobreescritura del método **toString()**.



- La clase **Persona** sobreescribe **complementamente** el método. La clase **Estudiante**, sobreescribe **parcialmente** el método.

Sobreescritura de métodos

Object

```
public String toString() {  
    //Implementación por defecto clase Object  
}
```

Persona

```
public String toString() {  
    return "Nombre:" + nombre + " Edad:" + edad;  
}
```

Estudiante

```
public String toString() {  
    return super.toString() + " Cred:" + creditos;  
}
```

Profesor

```
public String toString() {  
    return super.toString() + " Dep:" + departamento;  
}
```

Sobreescritura de métodos

```
class AppEstudiantePersona {  
    public static void main (String[] args) {  
        Persona p = new Persona("Luís García", 20);  
        Estudiante e = new Estudiante("Luís García",20);  
        Profesor pr = new Profesor("Luís García", 20, "Informática");  
        System.out.println(p);  
        System.out.println(p.toString());  
        System.out.println(e);  
        System.out.println(e.toString());  
        System.out.println(pr);  
        System.out.println(pr.toString());  
    }  
}
```

```
Nombre: Luís García Edad: 20  
Nombre: Luís García Edad: 20  
Nombre: Luís García Edad: 20 Cred: 60  
Nombre: Luís García Edad: 20 Cred: 60  
Nombre: Luís García Edad: 20 Dep: Informática  
Nombre: Luís García Edad: 20 Dep: Informática
```

SALIDA POR PANTALLA

Polimorfismo en JAVA

Polimorfismo en JAVA

El polimorfismo permite a las clases hijas heredar y redefinir los métodos de sus clases padres y tratar a los objetos de esas clases de manera genérica, sin importar su tipo específico. Hay dos formas principales de polimorfismo en JAVA: el polimorfismo de sobrecarga y el polimorfismo de sobrescritura.

- El **polimorfismo de sobrecarga** se refiere a la capacidad de una clase para tener métodos con el mismo nombre, pero con diferentes argumentos.
- El **polimorfismo de sobrescritura** se refiere a la capacidad de una clase hija para redefinir o sobrescribir los métodos de su clase padre.

```
public class Ejemplo {  
    void suma(int a, int b) {  
        System.out.println("a+b="+(a + b));  
    }  
    void suma(double a, double b) {  
        System.out.println("a+b="+(a + b));  
    }  
    void suma(int a, int b, int c) {  
        System.out.println("a+b+c="+(a + b + c));  
    }  
}
```

SOBRECARGA

```
public class Padre {  
    void mensaje() {  
        System.out.println("Padre");  
    }  
}  
public class Hija extends Padre {  
    void mensaje() {  
        System.out.println("Hija");  
    }  
}
```

SOBRESCRITURA

Polimorfismo en JAVA

Tenemos una jerarquía de clases de 3 niveles, y en cada uno de ellos se redefine por completo el método **hacerRuido()**.

En este ejemplo, la clase "Animal" es la superclase principal, y las clases "Perro" es la superclase de la clase "Pitbull". A continuación se muestra un ejemplo de polimorfismo de 3 niveles.

```
class Animal {  
    public void hacerRuido() {  
        System.out.println("El animal hace un ruido");  
    }  
}  
class Perro extends Animal {  
    public void hacerRuido() {  
        System.out.println("El perro ladra");  
    }  
}  
class Pitbull extends Perro {  
    public void hacerRuido() {  
        System.out.println("El pitbull tiene un ladrido profundo");  
    }  
}
```

```
public static void main(String[] args) {  
    ArrayList<Animal> animales = new ArrayList<>();  
    animales.add(new Animal());  
    animales.add(new Perro());  
    animales.add(new Pitbull());  
    for (Animal a : animales) a.hacerRuido();  
}
```

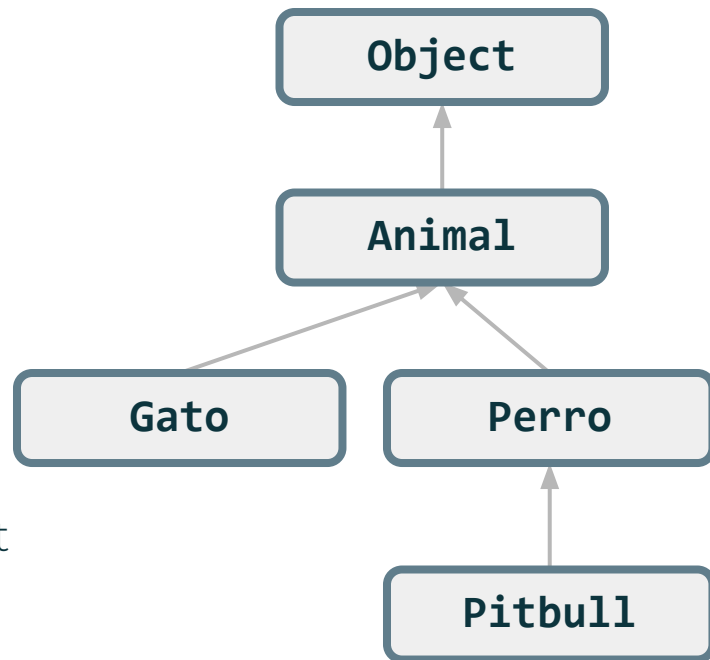
Clase Object en JAVA

Clase Object en JAVA

- **Toda clase en JAVA hereda implícitamente de la clase Object.** Ésta define los métodos que pueden ser invocados con cualquier objeto de JAVA.

Por ejemplo:

- `public String toString ( )`
- `public boolean equals (Object x)`
- Ejemplo de la jerarquía de clases:
 - Un Animal es un Object
 - Un Perro es un Animal y un Object
 - Un Gato es un Animal y un Object
 - Un Pitbull es un Perro, un Animal y un Object



Clase Object en JAVA

- La clase Object es una superclase predeterminada de todas las clases. Todas las clases de Java directa o indirectamente heredan de la clase Object.
- La clase Object define una serie de métodos que están disponibles en todas las clases en JAVA. Podemos destacar los siguientes métodos:
 - **clone()**: Crea y devuelve una copia del objeto actual.
 - **equals(Object obj)**: Compara el objeto actual con el objeto dado y devuelve verdadero si ambos objetos son iguales.
 - **getClass()**: Devuelve la clase del objeto actual.
 - **hashCode()**: Devuelve un código hash único para el objeto actual.
 - **toString()**: Devuelve una representación de cadena del objeto actual.
- Además, como ya hemos visto, todas las clases en JAVA pueden sobrescribir los métodos de la clase Object. Por ejemplo, una clase puede sobrescribir el método toString() para proporcionar una representación de cadena personalizada del objeto.

Clase Object en JAVA

```
public class Punto {  
  
    private int x;  
    private int y;  
  
    public Punto(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
}
```

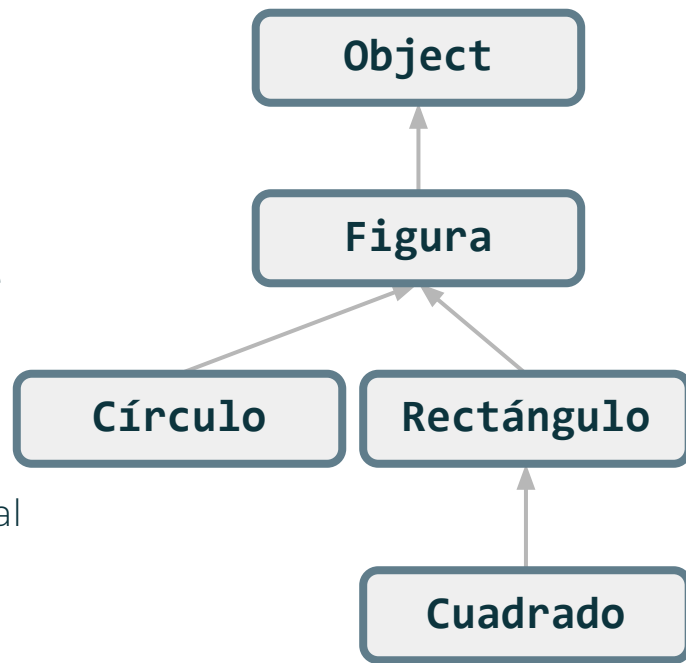
```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
    public String toString() {  
        return "Nombre: " + nombre + ", Edad: " + edad;  
    }  
}
```

```
public boolean equals(Object obj) {  
    if (obj == null) return false;  
    if (obj == this) return true;  
    if (!(obj instanceof Punto)) return false;  
    Punto otroPunto = (Punto) obj;  
    return this.x == otroPunto.x && this.y == otroPunto.y;  
}  
  
}
```

Clases abstractas en JAVA

Herencia en JAVA

- **Una clase abstracta es una clase que no se puede instanciar directamente**, y se utiliza como un modelo para crear otras clases que la extienden. Es decir, una clase abstracta sirve como una plantilla que define ciertos métodos y propiedades que se deben implementar en las clases que la extienden. Para que una clase sea abstracta, debe poseer por lo menos un método abstracto.
- Las utilizaremos cuando necesitemos crear una clase padre donde únicamente coloquemos una estructura muy general, dejando que sean las clases hijas quienes definan los detalles.
- Un método abstracto es un método vacío: un método el cual no posee cuerpo, por ende no puede realizar ninguna acción.
- La utilidad de un método abstracto es definir **qué** se debe hacer pero no el **cómo** se debe hacer.



Clases abstractas

- Veamos un ejemplo para que nos quede más claro. En este caso la clase posee una atributo, un constructor y un método, a partir de esta clase podremos generar todo tipo de figuras, ya sean cuadrados, rectángulos, triángulos, círculos etc...

```
public class Figura {  
    private int numeroLados;  
    public Figura(int numeroLados) { this.numeroLados = numeroLados; }  
    public double area() { return 0; }  
}
```

- Dentro de la clase encontramos el método área, método que se encuentra pensado para obtener el área de cualquier figura, sin embargo, cada figura posee su propia fórmula matemática para calcular su área. Por ello, todas las clases hijas tendrían que sobrescribir el método área e implementar su propia fórmula para así poder calcular su área.
- En estos casos, cuando la clase hija siempre debe sobrescribir el método, lo que podemos hacer es convertir dicho método en un método abstracto, un método que defina qué hacer, pero no cómo se deba hacer.

Clases abstractas

- Por tanto la clase Figura quedaría de la siguiente forma:

```
public abstract class Figura 2D {  
    private int numeroLados;  
    public Figura2D(int numeroLados) { this.numeroLados = numeroLados; }  
    public abstract double area();  
}
```

- Para heredar de una clase abstracta se hace del mismo modo que con clases no abstractas, basta con utilizar la palabra reservada `extends`.
- Al heredar de una clase abstracta es obligatorio implementar todos sus métodos abstractos, es decir debemos definir cómo se va a realizar la tarea.
- Desde una clase externa no podemos llamar al constructor de una clase abstracta. Sin embargo, las clases hijas pueden llamar al constructor de la clase abstracta a través de **`super()`**.

Clases abstractas

```
public class Triangulo extends Figura2D {  
    private Punto2D p1,p2,p3;  
    public Triangulo(Punto2D p1, Punto2D p2, Punto2D p3) {  
        super(3);  
        this.p1 = p1;  
        this.p2 = p2;  
        this.p3 = p3;  
    }  
    public double area() {  
        double a = Punto2D.distancia(p1, p2);  
        double b = Punto2D.distancia(p2, p3);  
        double c = Punto2D.distancia(p3, p1);  
        double s = (a + b + c) / 2;  
        return (Math.sqrt(s * (s - a) * (s - b) * (s - c)));  
    }  
}
```

```
public class Punto2D {  
    private double x, y;  
    public Punto2D(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
    public static double distancia(Punto2D p1, Punto2D p2) {  
        return Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));  
    }  
}
```

Ejercicios Clase Triángulo

A partir de las clases anteriores, Figura2D, Triángulo y Punto2D. Ampliar la clase triángulo para que incluya los métodos:

- **perimetro()**, deberá obtener el perímetro del triángulo, dicho perímetro se puede obtener a través de la suma de cada uno de los lados del triángulo.
- **esTriangulo()**, deberá comprobar si es un triángulo válido.
- **tipoTriangulo()**, deberá devolver el tipo de triángulo según sus lados. Los posibles valores serán NO_TRIANGULO, EQUILÁTERO, ISÓSCELES y ESCALENO.
- **toString()**, deberá mostrar toda la información del triángulo, coordenadas de los puntos, tipo, área y perímetro. Además se deberá redondear tanto el área como el perímetro mostrando únicamente 1 decimal. A continuación se muestra un ejemplo de salida del método.

```
El Triángulo es de tipo isósceles, de área 0.5 y perímetro 3.4  
Sus puntos son: p1[x=0.0, y=0.0], p2[x=1.0, y=0.0], p3[x=0.0, y=1.0]
```

Modificar el constructor para que al crear una instancia de triángulo automáticamente se guarde en la propia instancia el tipo de triángulo que forma según sus lados.

Interfaces en JAVA

Interfaces en JAVA

- La POO por medio de la herencia nos permite crear nuevas clases partiendo (o extendiendo) de otras.
- No obstante, JAVA no admite herencia múltiple. Una clase solamente puede heredar como máximo de otra.
- Para poder ofrecer una funcionalidad muy similar a lo que sería la herencia múltiple JAVA dispone de interfaces.
- Podemos definir las **interfaces** como una colección de métodos abstractos y propiedades constantes, sin implementación. Se especifica **qué** se debe de hacer pero no **cómo**, serán las clases hijas quienes definan el comportamiento.
- Las interfaces tienen las siguientes características:
 - **Todos los miembros son públicos** (no hay necesidad de declararlos públicos)
 - **Todos los métodos son abstractos** (A partir de JDK 8, hay métodos default)
 - **Todos los campos de datos son static y final**. Se usa para definir valores constantes.

Interfaces en JAVA

- Para crear una Interfaz:

```
public interface NombreInterfaz {  
    // constantes  
    public static final tipo nombreConstante = valorConstante;  
  
    // Métodos abstractos  
    public tipo nombreMetodo(parámetros);  
    public tipo nombreMetodo2(parámetros);  
    // ...  
}
```

- Para indicar que una clase implementa una interfaz:

```
public class nombreClase implements nombreInterfaz {  
    //implementación métodos  
    //...  
}
```

Si una clase implementa una interfaz deberá sobrescribir todos sus métodos

Interfaces en JAVA

- Veamos un ejemplo:

```
public interface Forma {  
    void dibujar();  
    double area();  
}
```

- Cómo podemos observar en la interfaz solo encontraremos métodos abstractos, métodos vacíos. Para poder implementar la interfaz basta con utilizar la palabra reservada **implements**.

```
public class Rectangulo implements Forma { ... }
```

En esencia las interfaces serán contratos que indicarán qué se debe hacer sin proveer ninguna funcionalidad.

Interfaces en JAVA

- Veamos un ejemplo:

```
public interface Forma {  
    void dibujar();  
    double area();  
}
```

```
public class Rectangulo implements Forma {  
    private double ancho;  
    private double largo;  
    public Rectangulo(double ancho, double largo) {  
        this.ancho = ancho;  
        this.largo = largo;  
    }  
    @override  
    public void dibujar() { System.out.println("Rectángulo dibujado"); }  
  
    @override  
    public double area() { return ancho*largo; }  
}
```

Interfaces en JAVA

- Veamos un ejemplo:

```
public interface Forma {  
    void dibujar();  
    double area();  
}
```

```
public class Circulo implements Forma {  
    private double radio;  
    public Circulo(double radio) {  
        this.radio = radio;  
    }  
  
    @override  
    public void dibujar() { System.out.println("Círculo dibujado"); }  
  
    @override  
    public double area() { return Math.PI*radio*radio; }  
}
```

Múltiples Interfaces

- Una interfaz puede extender a otra interfaz, lo que significa que hereda todos los métodos y constantes definidos en la interfaz padre.

```
public interface NombreInterfazHija extends NombreInterfazPadre {  
    // nuevos métodos y constantes  
}
```

- Una clase puede implementar múltiples interfaces separándolas con comas en la declaración de la clase.

```
public class nombreClase implements nombreInterfaz1, nombreInterfaz2 {  
    //implementación de los métodos de ambas interfaces  
    //...  
}
```

Múltiples Interfaces

- Veamos un ejemplo de implementación múltiple de 2 interfaces:

```
public class Cuadrado implements Forma, Color {  
  
    private double lado;  
    private String color;  
  
    public Cuadrado(double lado, String color) {  
        this.lado = lado;  
        this.color = color;  
    }  
    public double area()          { return lado * lado; }  
    public double perimetro() { return 4 * lado;      }  
    public String color()         { return color;      }  
}
```

```
public interface Forma {  
    public double area();  
    public double perimetro();  
}  
  
public interface Color {  
    public String color();  
}
```

Interfaces vs clases abstractas

- En las interfaces los atributos deben ser constantes. En una clase abstracta pueden ser de cualquier tipo.
- Una clase abstracta puede contener métodos que dispongan de implementación. Una interfaz sólo dispone de métodos abstractos.
- Como todos los métodos de una interfaz son abstractos no necesitamos incluir abstract en la definición de estos. En cambio, en una clase abstracta sí que necesitaremos hacerlo en aquellos métodos que lo sean.
- Una clase abstracta no es más que una clase común la cual posee atributos, métodos, constructores y por lo menos un método abstracto. Una clase abstracta no puede ser instanciada, solo heredada.
- Cómo JAVA no permite la herencia múltiple, habrá ocasiones en las cuales debamos utilizar interfaces, podemos verlas como contratos, contratos donde está establecido qué debe hacer la clase que la implementa.

Tipos enumerados en JAVA

Tipos enumerados

- En JAVA, las clases tipo enum (enumeración) se utilizan para definir un conjunto fijo de valores. Los valores definidos por una clase enum se denominan "constantes de enumeración". Las clases tipo enum se utilizan comúnmente para definir tipos de datos que sólo pueden tener un conjunto específico de valores.

```
public enum DiaSemana { //posibles valores }
```

- Características:**

- Sobre un tipo enum tenemos disponible el método `values()` que devuelve un array con todos los valores del enum en el orden en que son declarados.
- Un tipo enumerado puede añadir campos constantes al objeto enumerado y recuperar esos campos.
- No podemos crear objetos del tipo enumerado con la sentencia **new**.

```
public enum DiaSemana {  
    LUNES,  
    MARTES,  
    MIÉRCOLES,  
    JUEVES,  
    VIERNES,  
    SÁBADO,  
    DOMINGO;  
}
```

Tipos enumerados

- Las clases tipo enum también pueden tener métodos y constructores definidos. Por ejemplo, se puede agregar un método para obtener el número de días en un mes específico:
- La clase Mes define 12 constantes de enumeración que representan los meses del año. Además, la clase tiene un campo dias que se utiliza para almacenar el número de días en cada mes. La clase también tiene un constructor que se utiliza para inicializar el campo dias y un método getDias() que se utiliza para obtener el número de días en un mes específico.

```
public enum Mes {  
    ENERO(31),  
    FEBRERO(28),  
    MARZO(31),  
    ABRIL(30),  
    MAYO(31),  
    JUNIO(30),  
    JULIO(31),  
    AGOSTO(31),  
    SEPTIEMBRE(30),  
    OCTUBRE(31),  
    NOVIEMBRE(30),  
    DICIEMBRE(31);  
  
    private final int dias;  
    Mes(int dias) { this.dias = dias; }  
    public int getDias() { return dias; }  
}
```

Tipos enumerados

- En este ejemplo, se definen `diaActual` y `mesActual`, de tipo `DiaSemana` y `Mes` respectivamente. A continuación, se asignan los valores `JUEVES` y `ENERO` a partir de los nombres de las clases enumeradas.

```
public class AppTiposEnumerados1 {  
    public static void main(String[] args) {  
        String diasSemana = "";  
        DiaSemana diaActual = DiaSemana.JUEVES;  
        Mes mesActual = Mes.ENERO;  
        for (DiaSemana d : DiaSemana.values()) diasSemana += d.toString() + ", ";  
        System.out.println("Los días de la semana son: " + diasSemana);  
        System.out.println("Hoy es " + diaActual + " y el mes actual es " + mesActual);  
        System.out.println(mesActual + " tiene " + mesActual.getDias() + " días");  
    }  
}
```

Los días de la semana son: LUNES, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO,
Hoy es JUEVES y el mes actual es ENERO
ENERO tiene 31 días

SALIDA POR PANTALLA

Tipos enumerados

- En ocasiones puede que debamos recibir información por consola y almacenarla como tipo enumerado. Para ello, deberemos guardar en una String un valor compatible con algún valor de tipo enumerado en cuestión y luego hacer una conversión.
- Haremos dicha conversión a través del método **valueOf(String s)** que contienen todas las clases de tipo **enum**, dicho método recibirá una String con un posible valor de tipo enumerado. Si el valor de la String no es válido, se producirá un error en ejecución.

```
public class AppTiposEnumerados2 {  
    public static void main(String[] args) {  
        Scanner entrada = new Scanner(System.in);  
        System.out.println("Dime un mes " + Arrays.toString(Mes.values()));  
        String s = entrada.nextLine();  
        Mes mesActual = Mes.valueOf(s); // s → deberá ser un tipo enumerado válido  
        System.out.println(mesActual + " tiene " + mesActual.getDias() + " días");  
    }  
}
```

```
//Ejemplo para entrada → ABRIL  
ABRIL tiene 30 días
```

SALIDA POR PANTALLA

Comparar objetos : equals y hashCode

Comparar objetos: equals y hashCode

Los métodos `equals()` y `hashCode()` son importantes porque permiten que los objetos se comparen y se indexen correctamente en las colecciones de JAVA. La implementación correcta de estos métodos es esencial para garantizar que las colecciones funcionen correctamente y produzcan resultados esperados.

- **`equals()`**: se utiliza para comparar dos objetos y determinar si son iguales. Esta función es importante porque los objetos en JAVA se comparan por referencia, es decir, si dos objetos tienen la misma referencia (es decir, apuntan a la misma dirección de memoria), se consideran iguales. Sin embargo, en la mayoría de los casos, lo que se quiere es comparar el contenido de los objetos, en lugar de la referencia.
- **`hashCode()`**: se utiliza para obtener un valor numérico único que representa un objeto. Este valor se utiliza por el sistema de colecciones de JAVA para indexar y buscar objetos. Para implementar correctamente la función `hashCode()`, es necesario seguir las siguientes pautas:
 - Si la función `equals()` devuelve `true` para dos objetos, entonces los valores devueltos por la función `hashCode()` deben ser iguales.
 - Si dos objetos son diferentes según la función `equals()`, entonces sus valores devueltos por la función `hashCode()` pueden o no ser diferentes.

El método equals de la clase Object

- Para implementar correctamente la función equals(), es necesario sobrescribir el método de la clase base Object, dicha función se utiliza para determinar si dos objetos son iguales. Por lo que es importante que se implemente correctamente para que los objetos se comparen por su contenido y no por su referencia.
- Por otro lado, la implementación correcta de hashCode() debe garantizar que dos objetos que sean iguales según la función equals() devuelvan el mismo valor numérico, mientras que objetos diferentes pueden devolver diferentes valores.

```
public class Punto2D {  
    private double x;  
    private double y;  
    ...  
    public boolean equals(Object obj) {  
        if (obj == null) return false;  
        if (obj == this) return true;  
        if (getClass() != obj.getClass()) return false;  
        Punto2D otroPunto = (Punto2D) obj;  
        return x == otroPunto.x && y == otroPunto.y;  
    }  
}
```

Los números de punto flotante (float y double) se representan internamente en binario. Como ya vimos en la segunda unidad, esto puede provocar en ocasiones pérdidas de precisión.

Por esta razón, se recomienda utilizar métodos como **Double.compare()** para comparar valores de punto flotante en lugar del operador **==**

El método equals de la clase Object

SIN SOBRESCRIBIR EL MÉTODO EQUALS

```
Punto2D p1 = new Punto2D(0, 0);
Punto2D p2 = new Punto2D(0, 0);
Punto2D p3 = new Punto2D(0, 0);
Punto2D p4 = p1;

if(p1.equals(p2)) System.out.println("Iguales");
else System.out.println("Distintos");
//Distintos

if(p1.equals(p4)) System.out.println("Iguales");
else System.out.println("Distintos");
//Iguales

ArrayList<Punto2D> puntos = new ArrayList<>();
puntos.add(p1);
puntos.add(p2);
puntos.add(p3);
puntos.add(p4);

puntos.remove(new Punto2D(0, 0));
//no borra ningún punto
```

MÉTODO EQUALS SOBRESCRITO

```
Punto2D p1 = new Punto2D(0, 0);
Punto2D p2 = new Punto2D(0, 0);
Punto2D p3 = new Punto2D(0, 0);
Punto2D p4 = p1;

if(p1.equals(p2)) System.out.println("Iguales");
else System.out.println("Distintos");
//Iguales

if(p1.equals(p4)) System.out.println("Iguales");
else System.out.println("Distintos");
//Iguales

ArrayList<Punto2D> puntos = new ArrayList<>();
puntos.add(p1);
puntos.add(p2);
puntos.add(p3);
puntos.add(p4);

puntos.remove(new Punto2D(0, 0));
//borra el primer punto con coordenadas (0,0)
```

El método hashCode de la clase Object

- El hashCode de un objeto en JAVA **es un valor entero que se utiliza para identificar el objeto de manera única en una colección de objetos**. El hashCode se utiliza como una clave en la implementación de estructuras de datos como HashMap, HashSet y otras colecciones de Java.
- El hashCode se debe basar en el contenido del objeto, es decir, en el estado de sus campos.
- La función hash está diseñada para mapear la entrada de manera eficiente a un valor hash único y consistente. Esto lo hace especialmente útil en la búsqueda de datos y la optimización de rendimiento. Se utilizan en colecciones basadas en hash, lo veremos en profundidad en el tema 7.

```
public class Punto {  
    private int x; //1  
    private int y; //2  
    ...  
    public int hashCode() {  
        final int prime = 31;  
        int result = 1;  
        result = prime * result + x;  
        result = prime * result + y;  
        return result; //994  
    }  
}
```

```
public class Punto2D {  
    private double x;  
    private double y;  
    ...  
    public int hashCode() {  
        final int prime = 31;  
        int result = 1;  
        long temp;  
        temp = Double.doubleToLongBits(x);  
        result = prime * result + (int) (temp ^ (temp >>> 32));  
        temp = Double.doubleToLongBits(y);  
        result = prime * result + (int) (temp ^ (temp >>> 32));  
        return result;  
    }  
}
```